SkillNavora · Business AI Stack

COMPLETE GUIDE 04 OF 15

Webhooks

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

INTEGRATION · COMPLETE GUIDE

Webhooks: how apps talk to each other in real time

A webhook is an address that receives a message the moment something happens. Instead of an app asking 'anything new?' every minute, the other app calls your address when there is news. Webhooks power forms, payments, chatbots, and most automation tools.

1. The idea in one picture

A customer submits a form on your site. The site sends a small JSON message (an HTTP POST) to your webhook URL. Your automation receives it, does the work, and returns a status code. That is the whole mechanism.

```
POST https://hooks.example.com/abc123
Content-Type: application/json

{ "name": "Asha", "phone": "+91 98XXXXXXXX", "need": "Product management course" }
```

2. Anatomy of a request

Part	Purpose
Method	POST for sending data. GET is for reading.
URL	The unique address. Treat it like a password: anyone with it can send data to you.
Headers	Metadata, such as `Content-Type: application/json` and a signature header from the sender.
Body	The data, usually JSON.
Response	200 means received. 4xx means your request was wrong. 5xx means the receiver failed.

3. Build a webhook in n8n (worked example)

1. Add a **Webhook** trigger node, method POST, response mode *When last node finishes*.
2. Click **Listen for test event**, then send a test request with curl from your computer:

```
curl -X POST "YOUR_TEST_URL" -H "Content-Type: application/json" \
  -d '{"name":"Asha","phone":"+919800000000","need":"Course"}'
```

3. Add a **Google Sheets** node to save the row. Activate the workflow; the production URL now works.

4. Make sure a webhook is safe

- **Verify the sender.** Many services sign each request with a secret. Recompute the signature in a Code node and reject mismatches.
- **Validate input.** Check types and lengths; reject unexpected fields.
- **Respond quickly.** Acknowledge the request, then process the work, so the sender does not time out and retry.
- **Make it idempotent.** Senders retry on failure. Store each event ID and skip duplicates, so you do not send two welcome messages.
- **Rate-limit.** Limit repeated calls from one source (on your server, or with the platform's limits).

> **NEVER**
> Put a webhook URL in public code, a screenshot, or a shared document. If it leaks, rotate it.

5. Debugging checklist

- Did the sender actually send? Check its delivery log.
- Is the URL the production URL, and is the workflow active?
- Read the request body in the execution log; compare field names exactly (case matters).
- Check the status code returned.

6. Checklist

- Verified, validated, idempotent.

- Tested with a real and a malformed request.
- Alert set for failures.

Check the sending service's webhook documentation for its signature method and retry policy; these differ between services.